WOXSEN UNIVERSITY

SCHOOL OF TECHNOLOGY



AI-BASED DELIVERY ROUTE OPTIMIZATION SYSTEM

"An AI-Based System for Efficient Delivery Route Planning"

PROJECT-BASED LEARNING REPORT

Foundations of Artificial Intelligence

Course code: [25TU03MJM2]‖Semester: III |Academic year: 2026-2027

Submitted by

Pavan Tej - 25WU0101037

Vineel - 25WU0102299

Kushal Dev - 25WU0101058

Under the guidance of: Dr. Steven Raj

01 October 2026

DECLARATION

We declare that the project report titled "AI Campus Navigation and Routers: An AI-Based Route Finder for Intelligent Campus Travel" has been prepared for the Foundations of Artificial Intelligence course at Woxsen University. This report presents the system design, a reference route-search implementation, illustrative campus data, and a reproducible evaluation of that implementation.

The report separates implemented route-search behavior from proposed interface and deployment features. The campus graph used in the examples is a teaching dataset; it is not a surveyed map of Woxsen University. Sources used for the conceptual background are acknowledged in the references.

We are responsible for reviewing the final submission and confirming that it accurately represents the work submitted for the course. The faculty approval and signatures must be completed by the respective signatories.

Place: Hyderabad Date: 01 October 2026

Signatures of the candidates:

Pavan Tej - 25WU0101037 Signature: _____

Vineel - 25WU0102299 Signature: _____

Kushal Dev - 25WU0101058 Signature: _____

CERTIFICATE

This is to certify that the report titled "AI Campus Navigation and Routers: An AI-Based Route Finder for Intelligent Campus Travel" is submitted by the following students for the Foundations of Artificial Intelligence course at Woxsen University during the academic year 2026-2027.

Name	***Roll number***
Pavan Tej	25WU0101034
Vineel	25WU0102299
Kushal Dev	25WU0101058

The report covers campus graph modelling, destination resolution, A* route planning, accessibility and closure filtering, implementation design, and testing on illustrative data. It is submitted for review and verification by the concerned faculty member. Faculty guide name and designation: Dr. Steven Raj

Faculty signature: _____

Date: 01 October 2026

ACKNOWLEDGEMENT

We thank Dr. Steven Raj for guidance in connecting the concepts of intelligent agents and search algorithms with a practical campus navigation problem. The project helped us understand how a user request becomes a destination, how a campus can be represented as a graph, and how a route can be selected under different constraints.

We also thank the School of Technology, Woxsen University, for providing the academic setting for this project. We appreciate the discussions and feedback from classmates on clear route instructions, accessible paths, and the usefulness of a simple map interface.

Our team, Pavan Tej, Vineel, and Kushal Dev, is listed throughout this report with the updated roll numbers. The report brings together the problem definition, design, reference implementation, and evaluation so that the project can be explained consistently during submission and presentation.

The main learning from preparing the report is that a shortest route is useful only when the map is accurate, the destination is understood, and the selected walkways satisfy the user's needs. These concerns shaped the implementation and the future development plan.

Date: 01 October 2026

ABSTRACT

Finding a classroom, laboratory, library, hostel, or auditorium can be difficult for new students and visitors on a large campus. A static map shows locations but does not automatically choose a connected walking route or respond to a closed walkway. AI Campus Navigation and Routers addresses this problem through a goal-based route-finding system.

The campus is represented as a weighted graph. Buildings and junctions form nodes, while walkways form edges with distance, closure status, and step-free information. The user selects a starting point and destination, or enters a supported place name. A destination resolver maps the input to a location identifier. The route planner filters unavailable edges and applies A* search to return a minimum-distance path for the selected constraints.

The reference implementation is written in JavaScript. A binary min-heap manages the search frontier, and a Euclidean heuristic guides the search. Dijkstra's algorithm is obtained by setting the heuristic to zero. The report includes illustrative map and interface diagrams, code extracts, route traces, and a defined evaluation method.

Tests on a ten-node teaching graph verify normal routes, step-free routing, walkway closure, an unreachable destination, and an already-reached destination. Both search modes match an independent all-pairs shortest-distance calculation across 400 start-destination cases covering four graph configurations. These checks validate the reference graph-search behavior; they do not measure field navigation or user travel performance.

The project demonstrates how search, state representation, constraints, and understandable route explanations can support campus travel. Future work includes a surveyed map, building-level indoor routes, location updates, multilingual input, and usability testing.

Keywords: Campus navigation, intelligent agent, A* search, weighted graph, route planning, accessibility.

TABLE OF CONTENTS

Section	*Page*
Declaration	2
Certificate	3
Acknowledgement	4
Abstract	5
Table of contents	6
List of figures	9
List of tables	10
1 INTRODUCTION	11
1.1 Background	11
1.2 Motivation and relevance	11
1.3 Scope of the report	12
1.4 How AI is connected to navigation	12
1.5 End-to-end user example	13
1.6 Learning outcomes	13
2 PROBLEM STATEMENT AND OBJECTIVES	14
2.1 Problem definition	14
2.2 Objectives	14
2.3 Assumptions and project boundaries	15
2.4 Success criteria	15
2.5 Functional and non-functional requirements	16
3 LITERATURE REVIEW AND BACKGROUND	17
3.1 Intelligent agents and state spaces	17
3.2 Weighted graph representation	17
3.3 A* and Dijkstra search	18
3.4 Heuristic selection	18
3.5 Browser interfaces and route explanations	19
3.6 Related approaches and project gap	19
3.7 Background sources and relevance	20

TABLE OF CONTENTS (CONTINUED)

Section	*Page*
4 METHODOLOGY AND SYSTEM DESIGN	21
4.1 Overall architecture	21
4.2 Agent and environment model	21
4.3 Campus data and state	22
4.4 Destination resolution	22
4.5 Route-search process	23
4.6 A* pseudocode and heuristic conditions	23
4.7 Constraints and edge cases	24
4.8 Data flow through one request	24
4.9 Worked route example	25
4.10 Distance and estimated walking time	25
4.11 Evaluation design	26
4.12 Interface consistency and map maintenance	26
5 IMPLEMENTATION	27
5.1 Modules and browser integration	27
5.2 Graph construction and place lookup	28
5.3 A* route-search code	29
5.4 Priority queue and result integration	30
5.5 Map and workflow illustrations	31
5.6 Input and result interface designs	32
5.7 Search trace and rerouting illustrations	33
6 RESULTS AND DISCUSSION	34
6.1 Verified reference outputs	34
6.2 Comparing A* with Dijkstra	34
6.3 Reproducibility and test coverage	35
6.4 Error handling and accessibility	35
6.5 Interpretation and limitations	36
6.6 Practical value and deployment needs	36

TABLE OF CONTENTS (CONTINUED)

Section	*Page*
7 CONCLUSION AND FUTURE SCOPE	37
7.1 Conclusion	37
7.2 Future scope	37
REFERENCES	38
Reference notes and project provenance	39
APPENDIX A REPRODUCIBLE EVALUATION PLAN	40
APPENDIX B DESIGN DECISIONS AND QUESTIONS	41
B.1 Decisions to settle before deployment	41
B.2 Model and real-campus boundaries	41

Project team

Name	*Roll number*
Pavan Tej	25WU0101037
Vineel	25WU0102299
Kushal Dev	25WU0101058

Report title: AI Campus Navigation and Routers. Submission date: 01 October 2026.

LIST OF FIGURES

Section	*Page*
Figure 1. Illustrative campus graph and selected route	31
Figure 2. Navigation request and feedback workflow	31
Figure 3. Navigation input interface design	32
Figure 4. Step-free route result interface design	32
Figure 5. A* expansion trace for Main Gate to Library	33
Figure 6. Rerouting after a walkway closure	33

The diagrams are prepared for this navigation report. The campus map is illustrative and uses the same synthetic data as the reference route-search implementation. Interface figures show a proposed layout rather than screenshots of a deployed campus service.

LIST OF TABLES

Section	*Page*
Table 1. Background sources and their relevance	20
Table 2. System architecture layers	21
Table 3. Agent state and campus data	22
Table 4. Reference campus walkways	25
Table 5. Navigation module responsibilities	27
Table 6. Verified route outputs	34
Table 7. A* and Dijkstra expansion counts	34
Table 8. Verification coverage and outcomes	35
Table 9. Reproducible evaluation scenarios	40

Distance values are in example metres. Expansion counts are measured in the tested reference implementation and include the starting node and, when reachable, the goal node. Walking times are estimates derived from a stated speed assumption.

1 INTRODUCTION

1.1 Background

A university campus contains many destinations connected by walkways, roads, entrances, stairs, and junctions. Students may need to reach classrooms and laboratories quickly, while visitors may need help finding administrative offices or an auditorium. Knowing the name of a building does not automatically reveal a practical walking route from the current position.

Campus navigation is a useful Foundations of AI problem because it connects a clear goal with a representation of the environment and a sequence of actions. The goal is to reach a selected destination. The environment is a network of known campus locations and permitted walking connections. The actions are movements along those connections. A search procedure selects a route that satisfies the goal and the stated constraints [1], [2].

The proposed system treats buildings and junctions as graph nodes. Walkable connections are edges, each with a nonnegative distance. A route is a sequence of adjacent nodes, and its distance is the sum of the selected edge weights. A disconnected pair of locations must produce a clear failure message instead of an invented route.

1.2 Motivation and relevance

A route finder can reduce the effort required to interpret a static campus map. It can also explain why a familiar shortcut is unavailable when a path is closed or contains stairs. These features are particularly useful for first-time users and for users who request step-free travel.

The project focuses on a transparent search-based design. A user can inspect the selected path, distance, and constraint message rather than receiving an unexplained suggestion. This makes the system easier to demonstrate, test, and improve in an academic setting.

1.3 Scope of the report

This report covers the problem definition, campus graph design, navigation agent, destination lookup, route-search algorithm, implementation modules, illustrative interface, and reproducible evaluation. Its technical core is a JavaScript reference implementation of A* with a binary heap, together with a Dijkstra comparison obtained by disabling the heuristic.

The demonstration dataset contains ten locations and sixteen bidirectional walkways. Coordinates and distances are chosen for teaching the algorithm. They should not be used as actual distances or directions on the Woxsen campus. Replacing the teaching graph with verified campus data is a separate deployment step.

Users specify a starting point rather than relying on a live location sensor. The current design supports a single requested destination, distance-based routing, a step-free option, and temporary edge closures. A place resolver accepts configured names and aliases. It does not interpret every possible sentence or perform unrestricted conversational understanding.

1.4 How AI is connected to navigation

The AI component is goal-directed search. The system takes the goal location, reasons over legal transitions, evaluates candidate partial routes, and returns a path. A* uses both the distance already travelled and an estimate of the remaining distance. This is classical artificial intelligence and does not require a trained neural network [3].

Name lookup is a supporting interface function. It maps terms such as "lib" to the Library identifier. Machine learning or a language model could later expand the input interface, but the core route calculation should still use verified graph data and enforce the route constraints.

The map renderer communicates the reasoning result to the user. It is separate from the search procedure so that changing the visual design does not change the path distance or accessibility decisions.

1.5 End-to-end user example

Consider a user at the Main Gate who wants to visit the Library. The interface identifies the start as G and the destination as L. It reads the current map version and filters connections according to the selected preferences. The planner then searches the remaining graph and returns a route.

On the teaching graph, the ordinary shortest path is G > C > L: Main Gate, Cafeteria, Library. Its distance is 170 + 110 = 280 example metres. Two links on this route are marked as containing steps. A user who requests step-free travel therefore receives G > A > J > L, with a distance of 110 + 110 + 130 = 350 example metres.

If the Cafeteria-Library walkway closes, the same 350-metre alternative is selected for an ordinary request. The result panel can explain that the previous shortcut is closed. The original route is not kept merely because it was found earlier; the search must use the current graph snapshot.

1.6 Learning outcomes

The example shows that finding a path and defining a useful route are connected but separate tasks. The graph establishes where travel is possible. The cost function establishes what the system minimizes. Constraint filtering establishes which edges the user may use. A* then solves the remaining route-search problem.

The project also develops practical skills in graph representation, priority queues, heuristic reasoning, validation, and user communication. A believable navigation interface must show the same route that the planner calculated. If a diagram highlights one path while the written instructions list another, the system has failed even when the search function returns an optimal distance.

The important learning outcome is a testable connection between input, state, search, output, and feedback. Each stage has a defined responsibility and can be checked independently.

2 PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem definition

The campus has multiple known locations linked by walkways. A user supplies a starting location and a destination, with an optional step-free preference. Some walkways may be temporarily closed. The system must return a connected route of minimum total distance among the edges that remain permitted.

The central problem is therefore: how can an intelligent navigation agent use a campus graph to find a valid, understandable route while responding to user constraints and map updates? A route must be legal before it can be described as short. A visually direct line between two buildings is not sufficient evidence of a traversable walkway.

2.2 Objectives

• Represent campus buildings, junctions, and walkways using a weighted graph with stable location identifiers.

• Resolve supported location names and aliases without silently guessing an unknown destination.

• Use A* search to compute the least-distance route for the current graph and selected constraints.

• Support step-free filtering and temporary walkway closures before route search begins.

• Display a route sequence, total distance, estimated walking time, and understandable directions.

• Compare A* results with Dijkstra search and an independent shortest-distance calculation.

• Test disconnected locations, identical start and destination, invalid input, and changed route availability.

2.3 Assumptions and project boundaries

The reference map is finite and fully known to the planner. Every edge has a positive example distance, and all listed walkways are bidirectional. The graph does not model moving crowds, traffic signals, indoor elevators, or changes in access permissions during a single search. These can be introduced later if reliable data are available.

Each node has planar coordinates in the same example metre units as the edge weights. The straight-line distance between connected nodes is no greater than the stored walkway distance. This property supports the Euclidean heuristic used by A*. Coordinates are not latitude and longitude, and the demonstration is not connected to a GPS service.

The step-free field indicates that a sample walkway contains no steps in the teaching model. It does not certify a real route as wheelchair accessible: slope, surface quality, width, kerbs, doors, and operational lifts would also need assessment. The interface should use careful labels until those details are verified.

2.4 Success criteria

A successful route must begin at the selected start, end at the destination, and use only edges present in the filtered graph. Its reported distance must equal the sum of those edges. An unreachable goal must return an explicit status and an empty path. An already-reached destination must return a one-node path and zero distance.

For the reference search, optimality is checked by matching the returned distance to an independent all-pairs calculation. This checks numerical correctness on the sample graph; it does not establish map accuracy or user navigation quality in the field.

For a complete campus service, additional success criteria would include accurate location data, understandable instructions, correct response to closures, usable controls, and a map that remains consistent across screen sizes. Those are deployment and usability checks beyond the tested search core.

2.5 Functional and non-functional requirements

Functional requirements

• Choose a start and destination from the available campus location list.

• Accept exact configured names, identifiers, and aliases; ask the user to select when input is ambiguous.

• Read the current closure set and requested route preference before creating the search graph.

• Compute and reconstruct the route, or return a clear no-route message.

• Show the route on the map and provide the corresponding sequence of landmarks.

• Recalculate the route when the start, destination, preference, or map version changes.

Non-functional requirements

Correctness is the main requirement: the route, distance, and displayed instructions must agree. The graph model and search function should be modular so they can be validated without the interface. The map dataset should have an update timestamp and documented ownership before real deployment.

Usability requires visible labels, keyboard-friendly controls, readable contrast, and error messages that tell the user how to proceed. The input form should not erase the current selection after an invalid query. Route descriptions should remain available as text even when the map cannot be viewed.

The prototype can run with a local campus dataset and need not collect personal movement history. A future location-enabled version should request location access only when required and explain how the position is used.

Requirement boundary

The reference implementation verifies the search behavior and input lookup. The report specifies the remaining interface and deployment requirements so that a later application can implement them without changing the tested route contract.

3 LITERATURE REVIEW AND BACKGROUND

3.1 Intelligent agents and state spaces

An intelligent agent connects observations with actions in pursuit of a goal. The agents-and-environments framework is appropriate for navigation because the same movement has a different value depending on the destination and the available map [1]. For this project, the agent observes the chosen endpoints, graph snapshot, and route preference; it produces a route recommendation.

A state-space formulation describes the current location and the legal transitions to neighboring locations. The search starts at one state and aims to reach a goal state. Once the map and constraints are fixed, a state can be represented by its node identifier. A predecessor record allows the final sequence of transitions to be reconstructed.

3.2 Weighted graph representation

The graph is G = (V, E), where V is the set of locations and E is the set of walkways. An adjacency list stores the outgoing connections for each node. Each connection records its destination and weight. A bidirectional walkway creates one adjacency entry in each direction.

A weighted graph matters because the fewest connections need not form the shortest walking route. A three-edge route may be shorter than a two-edge route if its individual walkways are short. Breadth-first search is suitable for minimizing hop count in an unweighted graph; it is not generally suitable for minimizing the varied distances used here [2].

The cost of a route is the sum of its edge weights. The current project minimizes example walking distance. If the objective changes to travel time or perceived effort, both the edge costs and the heuristic must be reconsidered. A route described as "best" should always identify the criterion being minimized.

3.3 A and Dijkstra search*

Dijkstra search selects the frontier entry with the smallest accumulated path cost. On a finite graph with nonnegative weights, it returns a minimum-cost route when the goal is selected for expansion. It does not use a goal-directed estimate [2]. In the reference implementation, this mode is produced by setting the heuristic to zero.

A* assigns a priority f(n) = g(n) + h(n). The value g(n) is the best known distance from the start to n, while h(n) estimates the remaining distance to the goal. The heuristic can direct the search toward promising locations without changing the distance objective when its required conditions are satisfied [3].

Both methods relax outgoing edges: if reaching a neighbor through the current node gives a smaller distance, the stored best cost and predecessor are updated. A priority queue chooses the next entry. Old queue entries must be ignored when a later entry contains a better cost for the same node.

3.4 Heuristic selection

The reference implementation uses the Euclidean estimate h(n) = sqrt((x(n)-x(goal))^2 + (y(n)-y(goal))^2). It measures straight-line distance in the same planar metre units as the sample edge weights. The estimate cannot include an arbitrary multiplier that would overstate the remaining distance.

For every sample edge (u, v), its weight is at least the straight-line distance between u and v. The triangle inequality then gives h(u) <= w(u,v) + h(v). This consistency property permits the graph-search implementation to finalize an expanded node without reopening it [3].

Filtering out stair links or closed links preserves that inequality for the remaining edges. If future costs include discounts, mixed units, or coordinates with an unsuitable projection, the assumption must be checked again. Setting h = 0 is a safe fallback when no justified spatial heuristic is available.

3.5 Browser interfaces and route explanations

A browser interface can render campus nodes, edges, selected routes, and user controls. The Canvas API supplies drawing primitives for such a map, while normal HTML controls can provide the location selection and textual result panel [4]. The renderer should take the planner's output as data rather than calculate an independent route. JavaScript classes and modules provide a way to separate the map model, search algorithm, lookup function, and presentation code [5], [6]. This separation allows a route to be tested directly with known inputs. It also makes it easier to update a map label without modifying the priority queue.

A useful explanation identifies the objective and constraints: for example, "Shortest distance among step-free open walkways." The route list and edge-distance sum support that statement. A* is not a language model and cannot infer that a walkway is open merely from the name of a building.

3.6 Related approaches and project gap

Printed maps and signboards provide location information but depend on the user to combine it into a route. A shortest-hop search treats every connection equally. A weighted route finder addresses varied walking distances. A heuristic search can also reduce the number of explored locations on suitable maps.

Indoor positioning, live GPS, congestion prediction, and language-model assistants solve additional problems. They are useful extensions, but they are not prerequisites for demonstrating graph-based planning. A clear search core and verified map should come before broader features.

The classroom project gap is a compact, explainable navigation example that combines a destination request, graph constraints, route search, output, and repeatable verification. The report addresses that gap with a reference algorithm and an illustrative interface rather than claiming an already deployed campus service.

3.7 Background sources and relevance

The sources below serve different purposes. The artificial intelligence text supports the state-space and search concepts. The browser documentation supports the planned software organization and map rendering. The test outputs later in the report provide the evidence for the particular reference implementation.

Table 1. Background sources and their relevance

Source	*Relevant idea*	*Use in this project*
Poole and Mackworth [1]	Agents and environments	Define observations, goal, and route recommendation.
Poole and Mackworth [2]	Graph search and path cost	Model locations, legal transitions, and route distance.
Poole and Mackworth [3]	A* and admissible heuristics	Choose the Euclidean estimate and state its conditions.
MDN Canvas tutorial [4]	Browser drawing	Draw nodes, walkways, and the selected path.
MDN JavaScript classes [5]	Software organization	Separate queue, planner, and map responsibilities.
MDN JavaScript Guide [6]	Language fundamentals	Organize arrays, maps, data validation, and controls.
MDN Geolocation API [7]	Device location access	Inform a future optional positioning extension.

A reference to a drawing API does not establish that the navigation logic is correct. Likewise, the theory of A* does not validate a particular campus dataset. Correctness depends on both a suitable algorithm and faithful data. The evaluation therefore checks returned distances independently and labels the example map clearly.

The report uses references to explain design choices. It reports only measured search outputs from the included teaching setup, without turning those outputs into a claim about actual campus travel.

4 METHODOLOGY AND SYSTEM DESIGN

4.1 Overall architecture

The system is organized into input, map preparation, reasoning, and output layers. The input layer records the user request. Map preparation supplies a consistent graph snapshot and removes prohibited edges. The reasoning layer computes a route. The output layer presents the same route as a map highlight and a sequence of instructions.

Table 2. System architecture layers

Layer	*Inputs or outputs*	*Responsibility*
Input	Start, destination, preference	Collect and validate a navigation request.
Map preparation	Nodes, walkways, closures	Build the permitted graph snapshot.
Reasoning	Path, distance, search trace	Run A* and reconstruct the route.
Output and feedback	Map, directions, status	Explain the result and request replanning after changes.

4.2 Agent and environment model

The reference environment is discrete, known, deterministic, and static during a search. A move is a transition along a listed edge. The navigation agent is goal-based because the selected destination determines when a route is complete. It recommends actions for a person; it does not physically move or control a vehicle.

A request can be represented as (start, goal, preference, graphVersion). Search state includes best known distance, predecessor, frontier, and expanded nodes. A closure or preference change creates a new request snapshot. The current search is completed on one snapshot to prevent inconsistent edge availability within a result.

4.3 Campus data and state

Each place has a stable identifier, a human-readable name, planar coordinates, and supported aliases. Every edge stores its endpoints, a positive distance, and a step-free flag. Closure information is maintained separately so a temporary shutdown does not permanently delete the walkway from the base dataset.

Table 3. Agent state and campus data

Field	*Meaning*	*Example*
start / goal	Selected location identifiers	G / L
nodes	Location names and coordinates	L: Library, (200, 120)
edges	Connections and distances	J-L: 130 example metres
stepFree	Whether the sample link has no steps	true for J-L
blocked	Closed undirected edge keys	C-L
best / parent	Lowest cost and predecessor	best[L] = 280; parent[L] = C
frontier / closed	Queued entries and finalized nodes	Binary heap / set
result	Status, path, distance, expansion trace	ok, [G, C, L], 280

4.4 Destination resolution

The resolver trims whitespace and converts the input to lowercase. It compares the result with configured identifiers, names, and aliases. For example, "lib" resolves to L, while "canteen" resolves to C. Empty or unknown text returns no matches and must not trigger a route to an arbitrary destination.

The function returns a list of matches. The current dataset has unique aliases, but a larger campus may have multiple libraries or laboratories. More than one match should cause the interface to ask the user to choose a building. This is safer and more understandable than silently accepting the first result.

4.5 Route-search process

After validating the endpoints, the planner places the start node in a min-heap with g = 0 and f = h(start). It repeatedly selects the entry with the lowest f value. If the entry is stale or already expanded, it is ignored. Otherwise, the node is finalized and its outgoing permitted edges are examined.

For each neighbor, the planner calculates candidate = g(current) + edgeWeight. When this is smaller than the neighbor's recorded best value, the distance and predecessor are updated and a fresh heap entry is added. Reaching the goal triggers path reconstruction through the predecessor chain.

4.6 A pseudocode and heuristic conditions*

```
A_STAR(graph, start, goal):


  best[start] = 0
  frontier.push(start, priority = h(start))
  while frontier is not empty:
    current = frontier.pop_min()
    if current is stale or already expanded: continue
    mark current as expanded
    if current == goal:
      return reconstruct(parent, start, goal)
    for each permitted edge current -> next:
      candidate = best[current] + weight(current, next)
      if candidate < best[next]:
      best[next] = candidate
      parent[next] = current
      frontier.push(next, candidate + h(next))
  return UNREACHABLE
```

The closed-set rule is valid here because the Euclidean heuristic is consistent with every sample edge distance. The program checks that each edge is at least as long as its straight-line separation. A different cost model requires a new check or the h = 0 fallback.

With a consistent heuristic and a binary heap, each permitted adjacency edge is considered when its source is expanded. Lazy duplicate entries can occupy O(V + E) space; queue operations give O((V + E) log(V + E)) time. On this simple graph, E <= V^2, so the familiar O((V + E) log V) bound also applies.

4.7 Constraints and edge cases

Constraint handling occurs before search. A closed walkway is removed from both directions of the adjacency list. For a step-free request, every edge marked as containing steps is also removed. The search then minimizes distance on the graph that remains; it does not calculate a route first and attempt to repair it afterwards.

An unreachable destination produces status "unreachable", an empty path, and a null distance. The interface should display "No route is available with the current settings" and retain the selected endpoints. It can offer a preference review, but it must not automatically override an accessibility request.

When the start equals the destination, the route is [start] and the distance is zero. Unknown identifiers cause a validation error. A negative or nonnumeric edge weight should be rejected during data import. Duplicate or mismatched node identifiers should also be caught before building the graph.

4.8 Data flow through one request

The request moves through six stages: collect input, resolve locations, validate identifiers, filter edges, run search, and display the result. The final object contains a status, path, distance, expansion count, and trace. Only a successful result produces a route highlight.

The route is not recalculated inside the renderer. Instead, the renderer follows the node sequence returned by the planner and draws the matching edges. A directions generator uses the same sequence to produce landmark instructions. A distance panel uses the recorded edge sum rather than measuring screen pixels.

If the user changes the preference or an administrator records a closure, the map version changes and a fresh result replaces the old one. A future server-backed application should include the version identifier in its route response to make this relationship explicit.

4.9 Worked route example

The teaching graph uses sixteen bidirectional connections. Nodes G, A, J, B, L, C, U, H, S, and R denote Main Gate, Administration, Central Junction, Academic Block, Library, Cafeteria, Auditorium, Hostel, Sports Ground, and AI Laboratory respectively.

Table 4. Reference campus walkways

Connection	*Example distance (m)*	*Step-free*
G-A	110	Yes
G-H	130	Yes
A-J	110	Yes
A-C	130	Yes
J-B	110	Yes
J-L	130	Yes
B-U	130	Yes
L-U	110	Yes
C-L	110	No
C-H	110	Yes
H-S	130	Yes
S-R	110	Yes
R-C	130	Yes
R-L	170	Yes
C-J	170	No
G-C	170	No

For G to L, the route G > C > L costs 170 + 110 = 280. The alternative G > A > J > L costs 110 + 110 + 130 = 350. Removing C-L, or requiring step-free travel, excludes the shorter route and leaves the 350-metre alternative.

4.10 Distance and estimated walking time

Estimated time = route distance / assumed speed. At 1.2 m/s, 280 m gives 233.3 seconds, approximately 3.9 minutes; 350 m gives 291.7 seconds, approximately 4.9 minutes. These are arithmetic estimates, not recorded journey times, and do not include stops, doors, queues, or individual mobility needs.

4.11 Evaluation design

The evaluation distinguishes route-search correctness from interface usability and map accuracy. The first can be checked on a fixed graph with known weights. The second requires users and a working interface. The third requires a campus survey and an update process. The report verifies the first and defines how the other two can be assessed later.

The reference program tests six named cases: a normal route, a step-free route, a closed shortcut, an isolated destination, a start equal to the destination, and a second ordinary destination. The expected distances are calculated from the graph. The test also checks alias resolution and an invalid node identifier.

For stronger numerical verification, a separate Floyd-Warshall calculation supplies the minimum distance for every ordered pair of locations. A* and Dijkstra are compared with that oracle on the base graph, the step-free graph, a graph with C-L closed, and a graph with all Library connections closed. Ten starts times ten goals times four configurations gives 400 cases.

4.12 Interface consistency and map maintenance

Each successful path is checked for the correct first and last nodes, legal adjacency at every step, and an edge sum equal to the returned distance. This guards against a correct numerical result with an incorrect predecessor chain. In a browser application, the highlighted route and directions should be checked against the same object.

A real map needs an owner, date, and method for recording closures. Changes should preserve stable identifiers and distinguish permanent geometry edits from temporary restrictions. A navigation result should identify the map snapshot on which it depends. Evaluation should also record errors explicitly. A no-route outcome can be correct when the destination is isolated. It should not be counted as an algorithm failure unless a permitted route actually exists. This distinction keeps the results interpretable.

5 IMPLEMENTATION

5.1 Modules and browser integration

The search core is implemented in JavaScript and executed against the teaching graph. It has no dependency on a map service or a machine-learning model. The browser interface design wraps this core with endpoint controls, a map renderer, directions, and a result panel.

Table 5. Navigation module responsibilities

Module	*Responsibility*	*Main output*
campusData	Store place records and base walkways	Nodes and edge records
placeResolver	Match configured names and aliases	Zero, one, or several place IDs
graphBuilder	Apply closures and step-free filter	Adjacency list
MinHeap	Order search entries by f and insertion order	Next lowest-priority entry
routePlanner	Search and reconstruct a route	Status, path, distance, trace
mapRenderer	Highlight the returned node sequence	Map illustration
directionsPanel	Present landmarks, cost, and messages	Text route explanation

A Find Route action first validates both endpoint selections. It then calls makeGraph with the current closure set and preference, calls search with the resolved IDs, and sends the result to the renderer. A result with status "unreachable" produces a message instead of a path.

The three figures pages following the code show the graph, workflow, interface layout, search trace, and closure response. These are report illustrations. The tested behavior described in Section 6 comes from the route-search program, not from a claim that a complete browser service has been deployed.

5.2 Graph construction and place lookup

The graph builder creates an empty adjacency list for every location. It skips closed edges and, when requested, links with steps. Every accepted bidirectional walkway adds two directed entries. The closure key sorts the endpoint identifiers so that C-L and L-C refer to the same walkway.

```
const edgeKey = (u,v) => [u,v].sort().join('-');
function makeGraph(blocked = [], accessible = false) {


  const closed = new Set(blocked);
  const graph = Object.fromEntries(Object.keys(nodes).map(id => [id,[]]));
  for (const [u,v,w,stepFree] of edges) {
    if (closed.has(edgeKey(u,v)) || (accessible && !stepFree)) continue;
    graph[u].push({to:v,weight:w}); graph[v].push({to:u,weight:w});
  }
  return graph;
}
function resolvePlace(text) {
  const q = text.trim().toLowerCase();
  if (!q) return [];
  return Object.entries(nodes).filter(([id,n]) =>
    [id,n.name,...n.aliases].some(s => s.toLowerCase() === q)
  ).map(([id]) => id);
}
```

The resolver returns matches rather than a guessed destination. For instance, resolvePlace("lib") returns ["L"]. Unknown input returns an empty list. A user-facing control should check the list length and either accept one location, request a choice, or explain that the place was not found.

The full sample data contain a name, coordinates, and aliases for each node. An edge record has the form [from, to, distance, stepFree]. Before importing real campus data, the application should validate identifiers, weight units, closure keys, and the heuristic condition. Import checks are separate from the reference search loop.

5.3 A route-search code*

The implementation below uses Euclidean h when useHeuristic is true. The same routine becomes Dijkstra search when useHeuristic is false. It records g, h, and f for every expanded node so that a result can be explained after the search.

```
function search(graph,start,goal,useHeuristic = true) {


  if (!nodes[start] || !nodes[goal]) throw new Error('Unknown location');
  const h = id => useHeuristic
    ? Math.hypot(nodes[id].x-nodes[goal].x,nodes[id].y-nodes[goal].y) : 0;
  const queue = new MinHeap(), best = {[start]:0}, parent = {};
  const closed = new Set(), trace = [];
  queue.push({id:start,g:0,f:h(start)});
  while (queue.size) {
    const cur = queue.pop();
    if (cur.g !== best[cur.id] || closed.has(cur.id)) continue;
    closed.add(cur.id);
    trace.push({id:cur.id,g:cur.g,h:+h(cur.id).toFixed(2),f:+cur.f.toFixed(2)});
    if (cur.id === goal) {
    const path = [goal];
    while (path[0] !== start) path.unshift(parent[path[0]]);
    return {status:'ok',path,distance:cur.g,expanded:trace.length,trace};
    }
    for (const e of graph[cur.id]) {
      const candidate = cur.g + e.weight;
      if (candidate < (best[e.to] ?? Infinity)) {
        best[e.to] = candidate; parent[e.to] = cur.id;
        queue.push({id:e.to,g:candidate,f:candidate+h(e.to)});
      }
    }
  }
  return {status:'unreachable',path:[],distance:null,expanded:trace.length,trace};
}
```

A stale heap entry is one whose stored g value is no longer the best recorded cost. Skipping it avoids using an outdated partial path. The closed set avoids repeat expansion under the consistent-heuristic assumption stated in Section 4.6.

When the goal is expanded, the predecessor chain is followed back to the start. If the frontier empties first, the routine returns an explicit unreachable status. The interface must branch on this status before attempting to display distance or directions.

5.4 Priority queue and result integration

The binary heap stores entries ordered by f. Equal priorities are resolved by insertion sequence, making the sample trace repeatable. Insertion moves an entry upward; removal places the last entry at the root and moves it downward until heap order is restored.

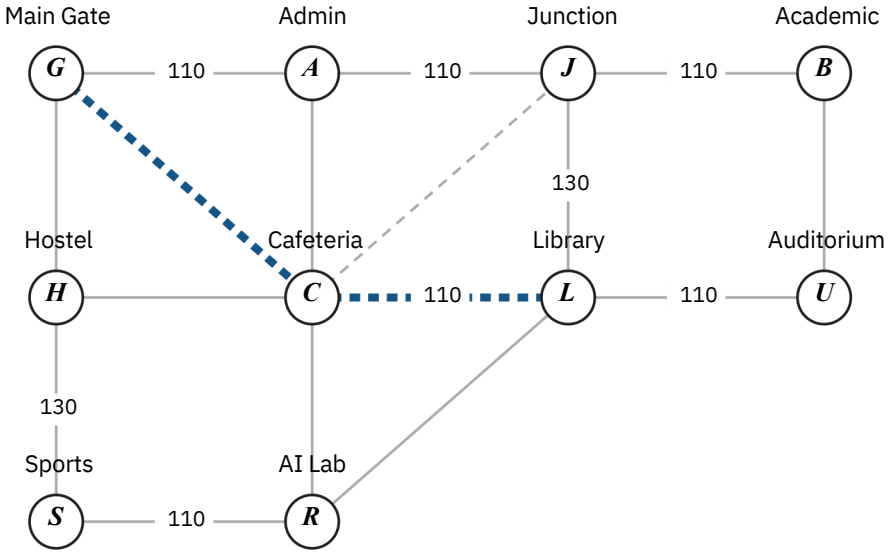
```
class MinHeap {

  constructor() { this.a = []; this.seq = 0; }
  less(x,y) { return x.f < y.f || (x.f === y.f && x.seq < y.seq); }
  push(item) {
    item.seq = this.seq++;
    const a = this.a; a.push(item);
    let i = a.length - 1;
    while (i > 0) {
      const p = (i - 1) >> 1;
      if (!this.less(a[i],a[p])) break;
      [a[i],a[p]] = [a[p],a[i]]; i = p;
    }
  }
  pop() {
    const a = this.a, first = a[0], last = a.pop();
    if (a.length) {
      a[0] = last; let i = 0;
      for (;;) {
        const l = 2*i+1, r = l+1; let m = i;
        if (l < a.length && this.less(a[l],a[m])) m = l;
        if (r < a.length && this.less(a[r],a[m])) m = r;
        if (m === i) break;
        [a[i],a[m]] = [a[m],a[i]]; i = m;
      }
    }
    return first;
  }
  get size() { return this.a.length; }
}
```

In the output layer, path identifiers are converted to place names and linked to the same coordinates used in the map. Distance comes directly from the successful search result. Estimated time is calculated only after the distance is known.

The UI should clear an old route after a failed request and show the new status. It should also recalculate when the graph or preference changes. These behaviors keep the route explanation consistent with the current inputs.

5.5 Map and workflow illustrations



Illustrative graph; example metres. Dashed links contain steps.

Figure 1. Illustrative campus graph with the 280-metre G-C-L route

The highlighted route follows listed walkways. Nodes represent buildings or junctions; dashed links contain steps. The map is schematic and the weights are example distances.

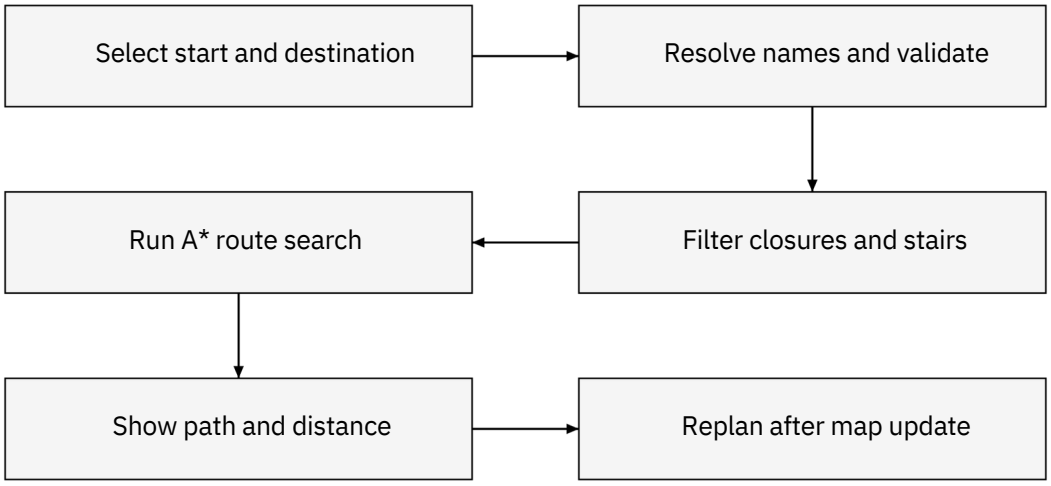


Figure 2. Navigation request and feedback workflow

Replanning repeats the validation, filtering, and search stages when an input or map condition changes. The output stage presents one consistent result object.

5.6 Input and result interface designs

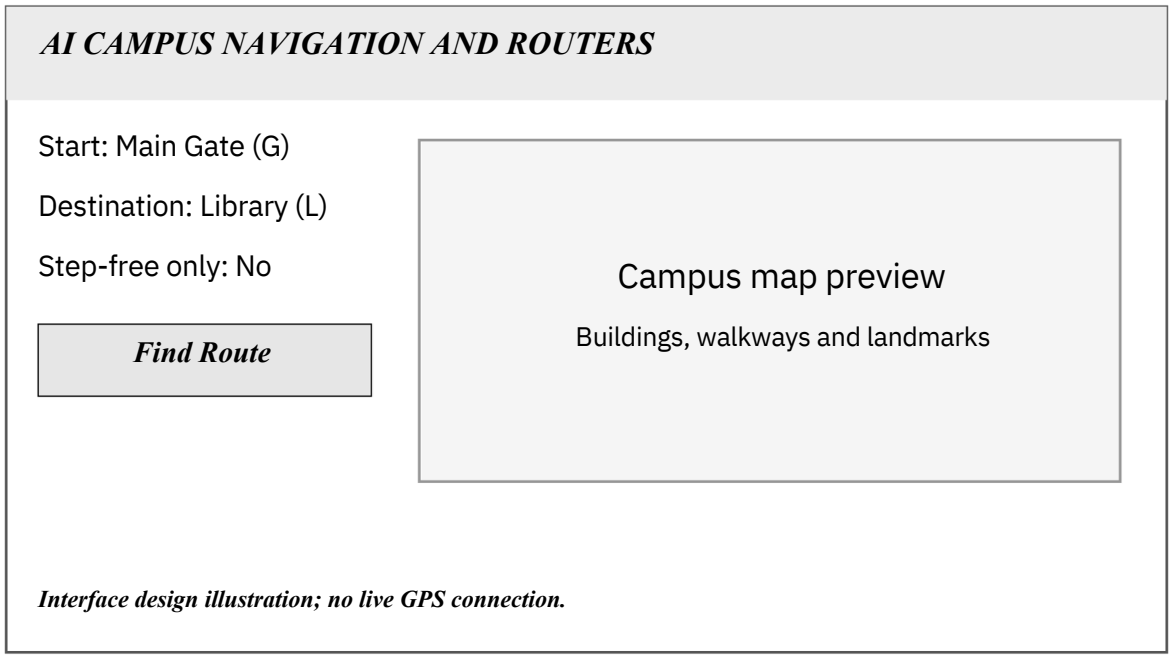


Figure 3. Navigation input interface design

The start, destination, and preference are visible together. A future browser implementation can allow list selection and configured alias lookup while keeping a textual result panel available.

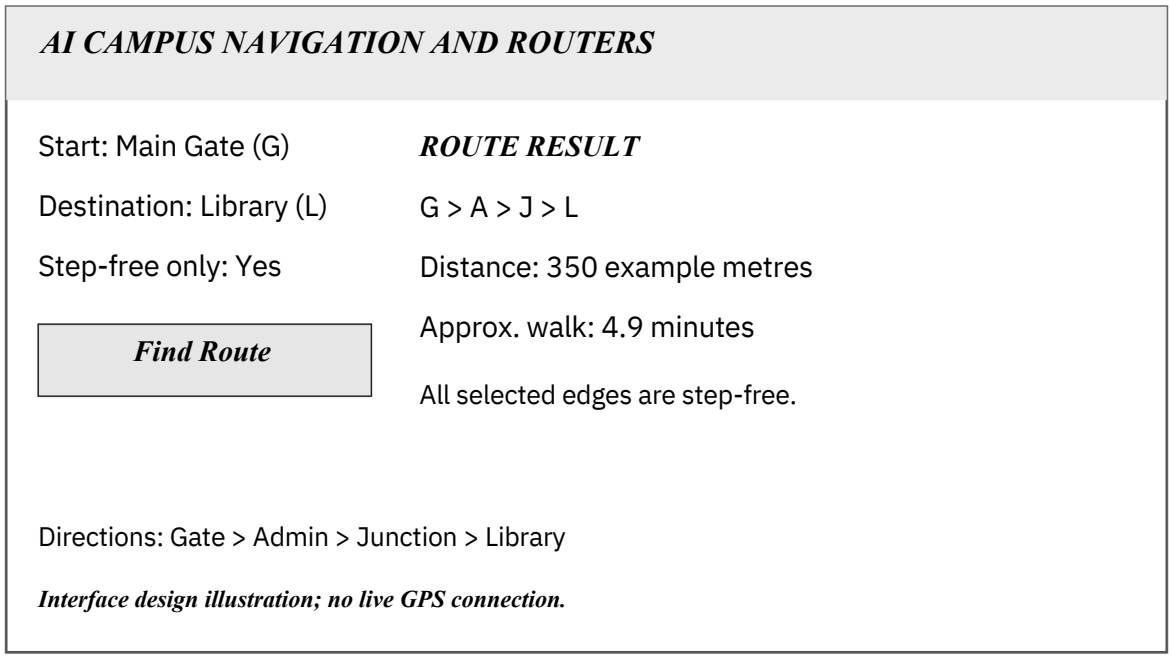


Figure 4. Step-free route result interface design

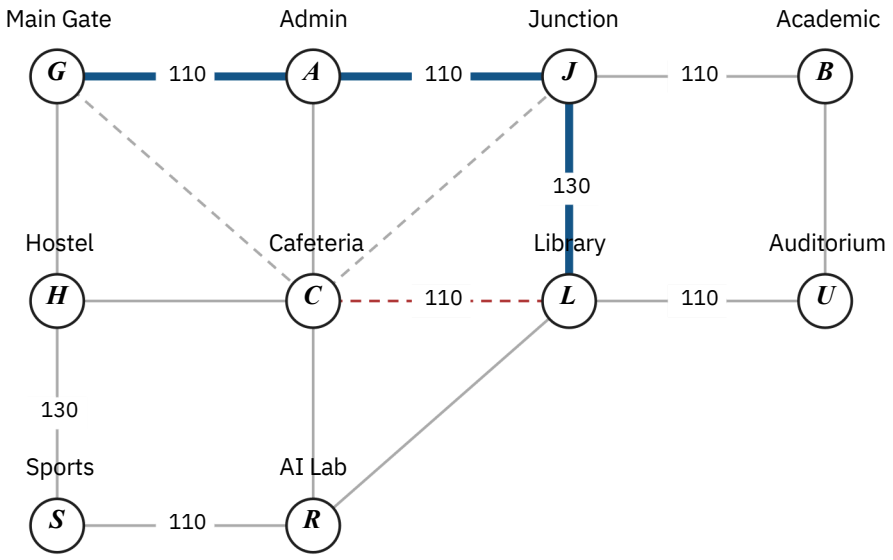
The result lists G > A > J > L and 350 example metres. The walking time of 4.9 minutes uses the declared 1.2 m/s assumption. This figure is a design illustration rather than a live location capture.

5.7 Search trace and rerouting illustrations

Expansion	*Node*	*g (m)*	*h (m)*	*f (m)*
1	G	0	233.24	233.24
2	A	110	156.2	266.2
3	C	170	100	270
4	L	280	0	280

Figure 5. Recorded A* expansion trace for Main Gate to Library

A* expands G, A, C, and L for this request. Node A is considered before C, although it does not belong to the final route. The algorithm returns the route when the Library is expanded with g = 280.



Illustrative graph; example metres. Dashed links contain steps.

Figure 6. Rerouting after the Cafeteria-Library walkway closes

The closed C-L link is excluded from the graph. The selected alternative is G > A > J > L, with distance 350. The same path satisfies the sample step-free preference. The added 70 metres results from the changed set of permitted routes.

6 RESULTS AND DISCUSSION

6.1 Verified reference outputs

The JavaScript reference search was executed on the documented teaching graph. The following route distances and statuses matched the expected values and the independent shortest-distance calculation. Results refer to synthetic data and do not describe measured walks on campus.

Table 6. Verified route outputs

Test	Scenario	A* path	Distance (m)	Check
T1	Gate to Library	G > C > L	280	Pass
T2	Step-free route	G > A > J > L	350	Pass
T3	Cafeteria-Library closed	G > A > J > L	350	Pass
T4	Library isolated	No route	-	Pass
T5	Already at destination	L	0	Pass
T6	Gate to Academic Block	G > A > J > B	330	Pass

6.2 Comparing A* with Dijkstra

Both modes return the same minimum distance for each case. A* expands fewer nodes in several examples because the heuristic guides the frontier. The counts include the starting node and a reachable goal; they exclude stale entries skipped before expansion.

Table 7. A* and Dijkstra expansion counts

Test	A* expanded	Dijkstra expanded	Minimum distance
T1	4	7	280 m
T2	6	8	350 m
T3	6	9	350 m
T4	9	9	Unreachable
T5	1	1	0 m
T6	4	9	330 m

The counts are specific to this graph, heuristic, and tie rule. They demonstrate a reduction in search work for these requests; they do not establish a general runtime improvement percentage or a travel-time saving for users.

6.3 Reproducibility and test coverage

The independent verification uses Floyd-Warshall over the filtered adjacency graph. It calculates distances without calling the A* routine or using its heuristic. For each of four graph configurations, all 100 ordered start-destination pairs are checked against both search modes.

Table 8. Verification coverage and outcomes

Check	*Coverage*	*Outcome*
Shortest distance	400 ordered-pair cases; A* and Dijkstra each compared with the independent oracle	All distances and unreachable outcomes agree.
Route reconstruction	Every successful A* result in the 400 cases	Correct endpoints, legal links, and matching edge sum.
Heuristic condition	All 16 base walkways	Distance at least straight-line separation.
Alias lookup	Configured alias, unknown name, empty input	Library found; unknown and empty input rejected.
Endpoint validation	Unknown start identifier Z	Validation error returned.
Named demonstration cases	Six scenarios in Table 6	Expected distance or status confirmed.

6.4 Error handling and accessibility

The isolated Library case correctly reports no route. The already-at-destination case returns [L] and zero distance. These outcomes are part of correct behavior rather than exceptions to be hidden from the interface.

The step-free case shows that filtering a short route can produce a longer valid alternative. A result should explain the selected constraint. It should not label a route fully accessible until real walkway geometry and access conditions have been reviewed. The verification covers the search core and configured lookup behavior. Browser event handling, screen-reader behavior, map display consistency, and live positioning remain part of the application integration tests.

6.5 Interpretation and limitations

The strongest result is numerical and functional: on the four teaching graph configurations, the reference implementation returns the same shortest distance as an independent calculation, and its successful paths use valid edges. The named scenarios also show understandable responses to route constraints and input boundaries.

A* does not guarantee that a user will arrive sooner in every real situation. The planner minimizes stored distance, while actual travel time can depend on crowds, waiting, weather, mobility, and access conditions. The estimated walking time is derived from a fixed assumed speed and is not a prediction trained on travel observations.

The heuristic helps only when it provides useful information in compatible units. A small graph may not produce a noticeable interface speed difference even when the expansion count decreases. A larger evaluation should measure repeated runtime under a declared environment, rather than converting expansion counts directly into a speed claim.

6.6 Practical value and deployment needs

A campus route finder can make directions more consistent by connecting destination names with permitted walkways and explicit preferences. A trace and distance explanation make its recommendations easier to inspect. Temporary closure filtering is especially useful when the available campus network changes.

Deployment requires a surveyed map, verified entrances, confirmed step-free links, and an administrator process for updates. Real-campus usability should be tested with new students and visitors. Tasks should include finding an unfamiliar building, responding to a closed link, and understanding a no-route message.

Indoor guidance and device positioning require additional engineering. GPS accuracy alone may not identify the correct doorway or floor. These capabilities should be added with their own validation rather than inferred from the success of the graph-search tests. The current report therefore provides a verified planning core, an implementation design, and a concrete next-step evaluation plan. It does not treat illustrative interface drawings as evidence of a field-tested navigation service.

7 CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

AI Campus Navigation and Routers applies classical AI search to a practical campus travel problem. It represents locations and walkways as a weighted graph, resolves a user's requested destination, filters route constraints, and computes an explainable minimum-distance path using A*.

The reference implementation demonstrates normal routing, step-free selection, closure-based replanning, unreachable destinations, and a zero-distance arrival case. Its A* and Dijkstra distances agree with an independent all-pairs calculation across 400 cases on four teaching graph configurations. Path reconstruction and edge-distance sums were also checked.

The main learning outcome is that useful navigation combines data, goals, constraints, and understandable output. The algorithm can only plan over the map it is given. Clear route explanations and a reliable map update process are therefore as important as the priority queue.

7.2 Future scope

• Replace the teaching graph with a surveyed campus map, including building entrances and verified walkway measurements.

• Add floor-level indoor routes, lift availability, and richer accessibility information.

• Integrate optional device positioning with clear accuracy messages and a manual-start fallback.

• Expand destination input with multilingual aliases and, if useful, a verified language-processing layer.

• Provide administrator controls for closures, map versions, and maintenance timestamps.

• Test interface usability, directions, and route understanding with representative campus users.

These extensions can build on the tested route-search contract. Each should introduce the data and checks required for its own claims, while preserving a transparent connection between the selected destination, permitted graph, and displayed result.

REFERENCES

[1] D. L. Poole and A. K. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3rd ed., "Agents and Environments."
https://artint.info/3e/html/ArtInt3e.Ch2.S1.html

[2] D. L. Poole and A. K. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3rd ed., "Graph Searching."
https://artint.info/3e/html/ArtInt3e.Ch3.S3.html

[3] D. L. Poole and A. K. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3rd ed., "Informed (Heuristic) Search."
https://artint.info/3e/html/ArtInt3e.Ch3.S6.html

[4] MDN Web Docs, "Canvas tutorial."
https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API/Tutorial

[5] MDN Web Docs, "Classes," JavaScript Reference.
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes

[6] MDN Web Docs, "JavaScript Guide."
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

[7] MDN Web Docs, "Geolocation API."
https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API

Online documentation consulted on 01 October 2026. These sources support the background and proposed interface features; the implementation results are from the report's reference teaching graph.

REFERENCE NOTES AND PROJECT PROVENANCE

How the sources are used

Reference [1] explains the agent-and-environment framing. References [2] and [3] support graph search, path cost, and heuristic reasoning. They are used to justify the navigation model and the conditions attached to A*. The report paraphrases these ideas and states its own implementation choices.

References [4], [5], and [6] describe browser drawing and JavaScript organization. They inform the proposed map interface and modular design. Reference [7] concerns a future reference

optional location extension; live geolocation is outside the current implementation.

Report materials

The supplied PDF provides the report format and institutional front matter. Its previous topic, team details, screenshots, and project-specific results have been replaced with campus navigation material. The six new figures are prepared for this report and describe the teaching graph, workflow, proposed interface, search trace, and closure response.

The reference dataset comprises ten sample locations and sixteen bidirectional walkways with coordinates, distances, and step-free flags. The verified outputs are generated by the JavaScript search program described in Section 5. The independent oracle calculates all-pairs distances on the same permitted graph.

Updated submission details

Name	*Roll number*
Pavan Tej	25WU0101034
Vineel	25WU0102299
Kushal Dev	25WU0101058

Topic: AI Campus Navigation and Routers. Date: 01 October 2026. Course: Foundations of Artificial Intelligence. Semester: III. Academic year: 2026-2027.

The certificate and declaration retain signature spaces for completion by the relevant students and faculty. No scanned or generated signature is included.

APPENDIX A REPRODUCIBLE EVALUATION PLAN

Freeze the node records, base edges, closure set, preference, heuristic rule, and tie-breaking rule before a test. Use the same filtered graph for A*, Dijkstra, and the independent distance oracle. Reset all per-search state between requests.

Table 9. Reproducible evaluation scenarios

Case	*Controlled setup*	*Expected check*
A	Base graph; G to L	Route G-C-L; distance 280.
B	Step-free graph; G to L	Route G-A-J-L; distance 350.
C	Close C-L; G to L	Alternate route; distance 350.
D	Close J-L, L-U, C-L, L-R	Library unreachable from G.
E	L to L	Path [L]; distance zero.
F	Base graph; G to B	Route G-A-J-B; distance 330.
G	Alias "lib"; unknown or empty input	One Library match; no unknown match.
H	Every pair on four configurations	400 cases match independent distances.

Recorded fields

Store the case identifier, map configuration, start, goal, preference, result status, returned path, distance, expansion count, and ordered g/h/f trace. For every successful path, verify legal adjacency and the sum of weights. For an unreachable goal, compare the outcome with the oracle.

Application and field evaluation

After integrating the browser interface, check that the map highlight, landmark list, distance, and status agree. Then conduct separate campus-map and user-task checks. Record actual journey times only during those field trials, with the route and conditions documented.

The search-core checks described in this report are completed on teaching data. Interface and field checks are proposed next steps. Keeping the records separate prevents a correct algorithm test from being mistaken for proof of a complete deployed service.

APPENDIX B DESIGN DECISIONS AND QUESTIONS

B.1 Decisions to settle before deployment

Who maintains the map? A real deployment needs a named administrative role and an update procedure. The map should record the verification date for entrances, walkways, and closures.

What does step-free mean? The teaching model excludes stair links. A campus service must also assess slopes, surfaces, kerbs, widths, doors, and lift operation before describing a route as accessible.

What cost is minimized? The reference system minimizes distance. Travel-time or effort routing needs new edge costs and a heuristic that does not overestimate that cost. Mixed distance and time units should not be added together.

How is the start established? The current system uses manual selection. A future position-enabled version should display uncertainty and allow the user to correct the start when a device estimate is wrong.

How are ambiguous destinations handled? Repeated building or room names require a selection step. A language interface should resolve a stable place identifier before invoking the route planner.

When must a route be recalculated? A changed start, destination, preference, closure set, or map version invalidates the old result. The application should visibly replace it and explain the new status.

B.2 Model and real-campus boundaries

The report demonstrates reasoning over a known symbolic graph. It does not verify live sensor accuracy, building access permissions, indoor positioning, or the physical condition of real walkways. Those require their own data collection and acceptance checks.

This boundary is useful for a Foundations of AI project: it makes the search assumptions explicit and the reported results repeatable. A surveyed campus map and a working browser interface can then extend the design while preserving the tested navigation contract.

Team Pavan Tej - 25WU0101037; Vineel - 25WU0102299; Kushal Dev - 25WU0101058. ***Report date:*** 01 October 2026